

第三章第二讲：时序逻辑、状态机与时序约束基础

Digital Circuit Getting Started

2026 年 5 月 23 日

1 本讲导读

上一讲回答的是“逻辑从哪里来”，这一讲要回答的是“系统为什么会因为时间而失败”。逻辑表达式写对、门级结构连对，并不能保证系统正确工作——这只有在没有历史、没有状态、也没有采样时刻约束的理想组合世界里成立。一旦引入寄存器、时钟与状态，系统语义就变了，正确性也不再只由逻辑关系决定。

寄存器不只是“能存 0 和 1 的盒子”，它们实际上在切分系统边界：哪些计算必须在一个时钟周期内收敛，哪些值会在边沿被采样，哪些错误会被传播到下一拍。状态机是“寄存器 + 组合逻辑”这种组织方式的一个标准例子，但它的意义只有放在时序计算的框架下才能充分展开。

建立时间、保持时间、时钟偏斜、时钟不确定性、亚稳态、同步边界——这些概念共同在解决一个问题：如何让离散时间上的数字抽象，在真实电路的连续物理世界里仍然保持可靠。它们不是术语表上的条目，而是同一套工程约束的不同侧面。看清这一点，后面的 RTL 建模、时序分析、CDC、流水线和系统级设计才有坚实语义基础。

2 Learning Objectives

读完这一讲，你应当能够：

- 说明组合逻辑与时序逻辑在系统语义上的根本差异。
- 区分 latch、flip-flop、register 与 register chain 的作用，并知道它们为什么存在。
- 用“状态寄存器 + 下一状态逻辑 + 输出逻辑”的框架解释最基本的状态机结构。
- 用“寄存器边界 + 组合逻辑锥”的视角观察同步数字系统。
- 理解 propagation delay、setup time、hold time、clock skew 与 uncertainty 在约束什么。
- 解释 setup violation 与 hold violation 的典型成因和后果。
- 对简单寄存器到寄存器路径计算最基本的 setup/hold 约束。
- 理解亚稳态是什么，为什么它不能被彻底消灭，只能被降低概率。
- 说明 clock domain crossing 的最小风险，以及双触发器同步器能解决什么、不能解决什么。

3 为什么这一讲值得学

引入触发器时，很容易进入定义堆叠：锁存器是什么、触发器是什么、`setup` 是什么、`hold` 是什么、`Moore/Mealy` 有什么区别。背下这些定义不算难，难的是在离开课本后仍然知道什么时候该担心什么问题。根源在于：如果没有先把主问题钉住，定义越多，认知越碎。这一讲的主问题其实很统一：一旦系统开始依赖状态和采样时刻，它要如何在真实时间里保持可靠？

寄存器的引入，让系统从“当前输入决定当前输出”的纯函数关系，变成“当前状态和当前输入共同决定下一状态和输出”的状态转移关系。这不仅让设计能力大幅提升，也让失败模式急剧变化。组合逻辑里一个短暂毛刺，在纯组合网络里也许转瞬即逝；但如果这个毛刺恰好在采样边沿附近进入寄存器，它就可能被正式写进系统状态。于是，逻辑、时间和状态不再彼此分离，而是被绑进同一个因果闭环里。

到第四章开始写 `always @(posedge clk)` 时，会知道那个时钟边沿在工程上意味着什么，会知道“寄存器边界”不是语法装饰，而是系统切分方式。进入综合与时序分析时，也不会把 `setup/hold` 当成离逻辑很远的后端术语，因为它们已经被放回了系统语义中心。在这一讲里先算一遍最基础的 `setup/hold` 约束，后面再接受更复杂的 STA 术语时，很多定义就不再悬空。

4 核心概念与对象

4.1 组合语义与时序语义的分界线

组合逻辑关心的是当前输入到当前输出的映射；时序逻辑则关心状态如何随着时间推进而演化。这个差别看似只是“多了个寄存器”，实际却改变了系统的解释方式。现在的输出，不再只由当前输入决定，还要看过去被保存下来的状态。也正因为有了状态，系统才第一次拥有“历史”：同样的输入，在不同状态下可能导致完全不同的输出与转移。

这也是为什么“寄存器边界”值得反复强调。它不是纯粹的实现细节，而是把系统切分成一个个离散时间步的关键边界。边界左边是上一个采样时刻已经确定的状态，边界中间是必须在本周期内完成的组合计算，边界右边则是下一次采样想要得到的新状态或输出。

4.2 Latch、Flip-Flop 与 Register

锁存器（`latch`）是电平敏感的，当使能有效时，输入变化可能直接继续影响输出；触发器（`flip-flop`）通常是边沿敏感的，只在时钟边沿附近采样。寄存器（`register`）则是一组并行触发器，用来一起保存多位状态。`latch` 像一个“门开着时可以通过”的存储单元，`flip-flop` 像“在离散瞬间拍一张照”，`register` 则是一排同时拍照的触发器。图中的主从触发器按真实结构复刻，在系统层面贡献了一套清晰的语义——边沿采样与时序窗口。

多个寄存器串起来，就形成寄存器链。它可以用于流水线切分路径，也可以用于同步跨域输入。寄存器链不只是几个方框排在一起；每多一级寄存器，系统就多出一个时间边界，也多出一次重新稳定和重新解释数据的机会。

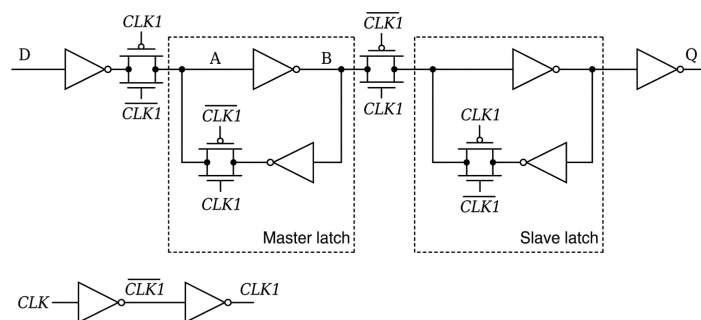


图 1: 按真实结构复刻的主从触发器示意。它建立了“电平敏感”和“边沿敏感”的语义差异，并为后续 setup/hold 窗口分析做准备。

4.3 寄存器边界与组合逻辑锥

同步数字系统最常见的结构视角，可以压缩成一句话：寄存器发射数据，组合逻辑做计算，寄存器捕获结果。这就是常说的寄存器到寄存器（register-to-register）路径。中间那片被组合逻辑覆盖的区域，常被描述为一个组合逻辑锥（logic cone），因为从多个输入源看向某个终点寄存器时，逻辑连接常常呈现出逐层汇聚的锥形结构。

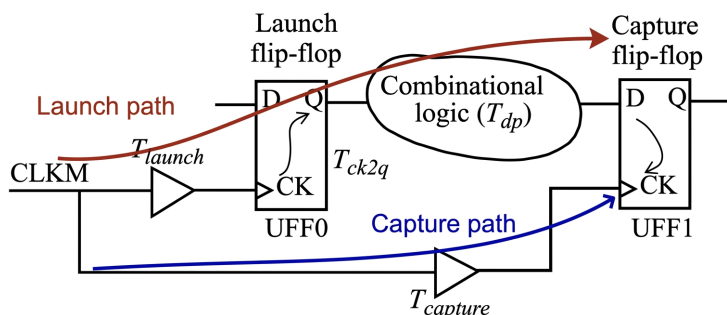


图 2: 寄存器-组合逻辑-寄存器的基本观察框架。时序约束、路径分析和流水线切分，都建立在这条骨架上。

这个视角非常重要，因为它把复杂系统里的很多判断都统一了。路径太长，setup 压力会变大；路径太短，hold 风险可能出现；逻辑锥太深，边沿前收敛可能来不及；跨域输入进入寄存器链，则是在重新定义同步边界。只要把系统先还原成寄存器边界和组合逻辑锥，很多后续术语就不再漂浮。

4.4 Setup/Hold: 同步系统的时间边界

建立时间（setup time）与保持时间（hold time）定义了触发器希望在采样边沿附近看到怎样的数据行为。前者要求数据在采样前提前稳定一段时间，后者要求数据在采样后继续保持一段时间。这不是厂商手册里“形式上列出来的参数”，而是触发器内部电路要可靠完成判决所需要的物理窗口。

当数据路径太慢、时钟周期太短、时钟到达差异太不利时，setup 违例就会出现：边沿来了，但正确数据还没到。反过来，当数据路径太快，或者终点寄存器看到时钟太晚时，hold 违

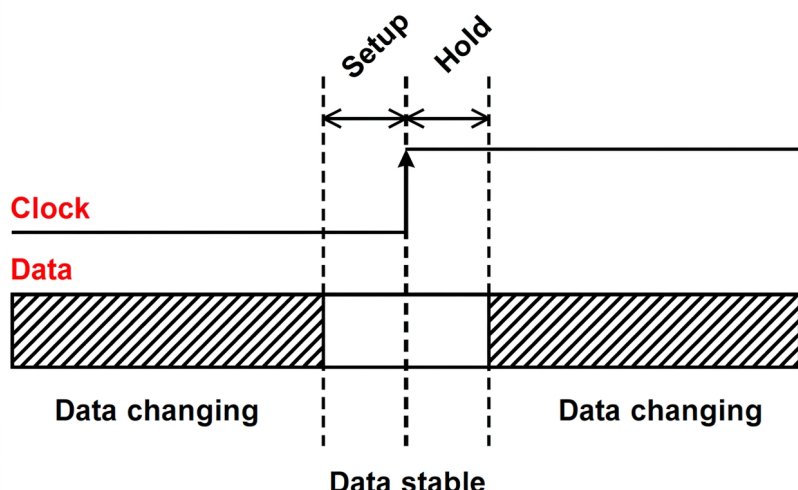


图 3: Setup/hold 窗口示意。触发器不是无限快、无限理想的采样器，它需要一个受保护的采样窗口。

例就可能出现：边沿刚过，新数据就抢着冲进来，污染了本应保持的旧数据。两类问题都和“时间窗口被破坏”有关，但破坏方式并不相同，因此修复方法也往往不同。

4.5 最小计算框架

知道 setup/hold 是什么之后，下一步是对最简单的寄存器到寄存器路径做延迟计算。先在零 skew、零 uncertainty 的简化情形下把骨架算式立住，再逐步加入 skew、uncertainty 和 jitter 这些修正项。

对一条寄存器到寄存器路径，最小 setup 检查可以写成：

$$T_{clk} \geq t_{clk \rightarrow q, max} + t_{comb, max} + t_{setup}$$

这里看的是真正的最长路径：起点寄存器在时钟边沿后把数据推出去，需要经历 $t_{clk \rightarrow q, max}$ ；数据再穿过最慢的组合逻辑路径，经历 $t_{comb, max}$ ；最后在终点寄存器采样前，还必须预留 t_{setup} 的稳定窗口。如果这三项之和已经超过一个时钟周期，那么 setup 就不满足。

最小 hold 检查则可以先写成：

$$t_{clk \rightarrow q, min} + t_{comb, min} \geq t_{hold}$$

这里看的恰好是最短路径：起点寄存器最快把新数据推出去，再经由最快的组合路径抵达终点寄存器；如果它到得太快，就会在终点寄存器还未结束保持窗口时把旧数据污染掉。

这两个式子把一套基本观察方式钉住了：setup 关注最长路径，hold 关注最短路径。后面加入 skew 与 uncertainty 时，是在这个骨架上继续修正，不是另起一套新逻辑。

4.6 Clock Skew 与 Clock Uncertainty

很容易把时钟想成一条同时覆盖全芯片的理想节拍线，仿佛每个寄存器都在同一瞬间听到同一拍子。现实并非如此。时钟本身也是一张物理网络，它要经过布线、缓冲、扇出和环境扰动才到达每个寄存器。于是，同一时钟边沿到达不同寄存器的时刻会有差异，这就是时钟偏斜 (clock skew)。再考虑抖动、建模误差和环境变化，就得到时钟不确定性 (uncertainty)。

这一点之所以重要，是因为很多 setup/hold 裕量其实不是纯粹由数据路径决定的，而是由“数据什么时候到”和“时钟什么时候被各个寄存器看到”共同决定的。换句话说，时钟本身也在参与时序问题，而不是站在系统外部发号施令。

方向感可以这样建立：对 setup 有利的 skew 会帮忙“借时间”，对 hold 不利的 skew 会让终点寄存器更晚关上保持窗口；uncertainty 则会进一步吞掉本来可用的裕量。更正式的 STA 章节会把这些修正项整理成完整的报告语言和约束语境。

4.7 状态机：寄存器系统的一个标准骨架

状态机是时序逻辑最标准、也最普遍的组织方式之一。它的最小骨架只有三部分：状态寄存器保存当前状态；下一状态逻辑根据当前状态和输入计算下一个状态；输出逻辑则根据状态，或者根据状态与输入共同决定对外输出。Moore 机与 Mealy 机的区别就落在这里：前者的输出更直接地挂在状态上，后者的输出还会更紧地受输入影响。

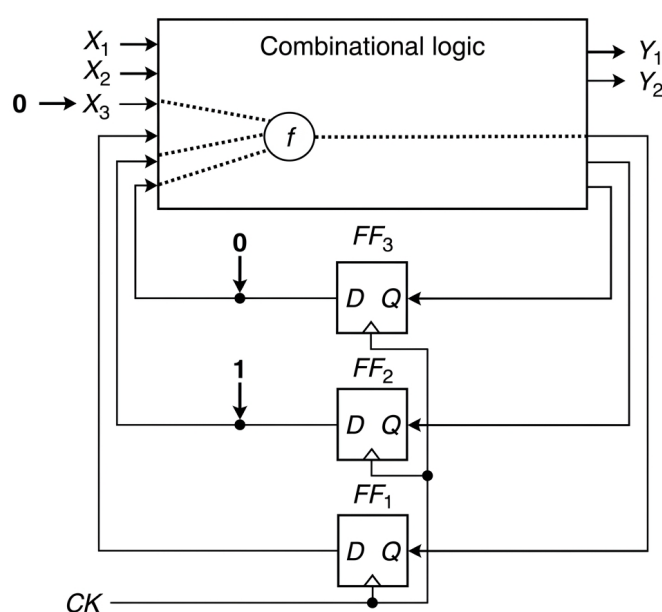


图 4：状态机结构示意图：一种寄存器与组合逻辑的组织方式。

一个基本判断比复杂的状态编码优先：什么属于状态、什么属于组合逻辑、什么只是输入条件。一旦写到 RTL 里，这三者没被清楚分开，问题往往比语法更难追踪。

4.8 亚稳态与同步边界

如果一个异步输入恰好在采样窗口附近变化，触发器内部可能一时无法决定该收敛到 0 还是 1，于是短时间停在一种不稳定的中间状态，这就是亚稳态。它不是第三种稳定逻辑电平，而是一种不稳定、可恢复、但恢复时间不可被完全保证上界的状态。

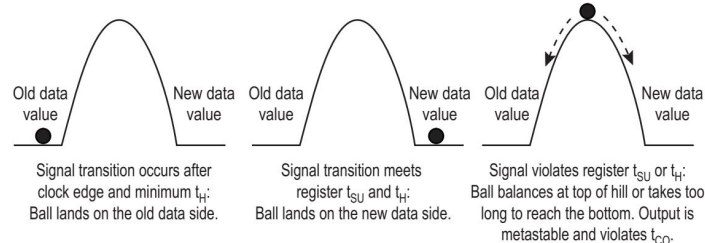


图 5: 亚稳态示意。关键不在于把它神秘化，而在于承认它是采样连续世界时不可彻底消除的概率事件。

工程上真正重要的结论是：亚稳态不能被彻底消灭，只能被降低概率。双触发器同步器的核心作用，就是给第一级可能进入亚稳态的触发器多一拍恢复时间，让第二级在更高概率上看到已经稳定的 0 或 1。但它只适用于单 bit 控制信号的最小同步场景；多 bit 总线、握手协议、异步 FIFO 之类的问题，还需要更完整的 CDC 设计策略。

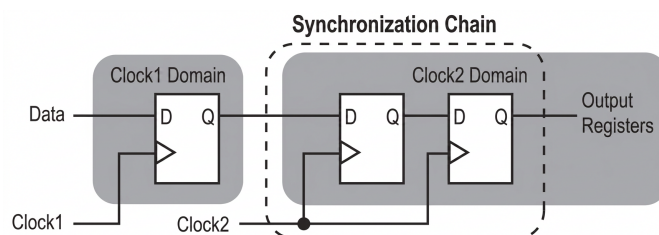


图 6: 双触发器同步器。它在解决概率问题，而不是自动解决所有跨域协议问题。

5 关键图示或案例

这一讲最值得反复看的图，不是某一页状态图本身，而是那张“寄存器-组合逻辑-寄存器”的骨架图。因为从那张图开始，**setup/hold** 是在检查什么、状态机的状态存在哪里、跨域风险会从哪里进入、为什么同步器要放在边界上——这些问题都开始有了统一的落脚点。面对一个新系统，先把它抽成几个寄存器边界和几片组合逻辑锥，再问状态、时序和跨域问题在哪里——这套观察方法比记住每一张单独的状态图更有用。

提示

四条线可以暂时放在心里：寄存器在制造时间边界，**setup/hold** 在保护采样窗口，**setup** 要看最长路径、**hold** 要看最短路径，异步输入会挑战这套秩序。后面的 RTL、STA 和 CDC 都会沿着这四条线反复回来加深。

警告

不要把“同步器”误解成万能胶。双触发器同步器只能降低单 bit 异步输入引发亚稳态传播的概率，不能自动保证多 bit 数据一致性，也不能替代协议设计、握手或异步 FIFO 之类更完整的跨域机制。

6 本讲小结

这一讲把系统从“函数”推进成了“受时间约束的寄存器系统”。寄存器引入了历史，也引入了时间边界；setup 和 hold 先通过最长路径/最短路径计算被立住，skew 与 uncertainty 再进一步修正这些边界；亚稳态和 CDC 则提醒我们，连续世界总会在同步边界上重新闯入数字系统。到这里，第三章真正完成了它的承重任务：既让你看到逻辑如何从器件世界里长出来，也让你知道一旦引入寄存器和时钟，系统正确性就必须同时受逻辑关系和时间关系约束。下一章进入 RTL 时，重点就可以自然转向：这些语义边界在 Verilog/SystemVerilog 里应该如何被正确建模。

7 练习题

1. 画出一个寄存器-组合逻辑-寄存器结构，并在图中标出数据路径、时钟路径、setup 检查窗口和 hold 检查窗口。
2. 对一个简单自动售货机或交通灯控制器，区分哪些信息属于“状态”，哪些逻辑属于“下一状态逻辑”，哪些属于“输出逻辑”。
3. 已知 $t_{clk \rightarrow q, max} = 120\ ps$ 、 $t_{comb, max} = 680\ ps$ 、 $t_{setup} = 80\ ps$ 、时钟周期 $T_{clk} = 900\ ps$ ，判断该路径 setup 是否满足；若不满足，还差多少裕量。
4. 已知 $t_{clk \rightarrow q, min} = 40\ ps$ 、 $t_{comb, min} = 20\ ps$ 、 $t_{hold} = 90\ ps$ ，判断该路径 hold 是否满足；若不满足，需要额外补多少最小延迟。
5. 说明为什么异步按键直接进入主时钟域寄存器会有风险；进一步解释双触发器同步器在这里能做什么、不能做什么。